

Experiment 07:

Part A: Logistic Regression

Step 1: Import required libraries

Code:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import sklearn as s
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import
accuracy_score, confusion_matrix, classification_report
```

Step 2: Load the dataset

Code:

```
df=pd.read_csv("/train.csv")
df.head()
```

Output:

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN	S

Step 3: Check dataset's basic information

Code:

```
print("Shape of Dataset :",df.shape)
df.info()
```

Output:

```
shape of dataset : (891, 12)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
#   Column          Non-Null Count  Dtype
---  -
0   PassengerId     891 non-null    int64
1   Survived        891 non-null    int64
2   Pclass          891 non-null    int64
3   Name            891 non-null    object
4   Sex             891 non-null    object
5   Age            714 non-null    float64
6   SibSp           891 non-null    int64
7   Parch           891 non-null    int64
8   Ticket          891 non-null    object
9   Fare            891 non-null    float64
10  Cabin           204 non-null    object
11  Embarked        889 non-null    object
dtypes: float64(2), int64(5), object(5)
memory usage: 83.7+ KB
```

Step 4: Display column names and missing values

Code:

```
print(df.columns.tolist())
print(df.isnull().sum())
```

Output:

```
***  ['PassengerId', 'Survived', 'Pclass', 'Name', 'Sex', 'Age', 'SibSp', 'Parch', 'Ticket', 'Fare', 'Cabin', 'Embarked']
PassengerId    0
Survived       0
Pclass         0
Name           0
Sex            0
Age           177
SibSp          0
Parch          0
Ticket         0
Fare           0
Cabin         687
Embarked       2
dtype: int64
```

Step 5: Select relevant features

Code:

```
features=['Pclass', 'Sex', 'Age', 'SibSp', 'Parch', 'Fare', 'Embarked']
data=df[features + ['Survived']].copy()
data.head()
```

Output:

	Pclass	Sex	Age	SibSp	Parch	Fare	Embarked	Survived
0	3	male	22.0	1	0	7.2500	S	0
1	1	female	38.0	1	0	71.2833	C	1
2	3	female	26.0	0	0	7.9250	S	1
3	1	female	35.0	1	0	53.1000	S	1
4	3	male	35.0	0	0	8.0500	S	0

Step 6: Handle the missing values**Code:**

```
data['Age']=data['Age'].fillna(data['Age'].median())
data['Embarked']=data['Embarked'].fillna(data['Embarked'].mode()[0])

print('Missing values after handling :')
print(data.isnull().sum())
```

Output:

```
*** Missing values after handling :
Pclass      0
Sex          0
Age          0
SibSp        0
Parch        0
Fare         0
Embarked     0
Survived     0
dtype: int64
```

Step 7: Convert categorical data into numerical form using One Hot Encoding**Code:**

```
data=pd.get_dummies(
    data,
    columns=['Sex','Embarked'],
    drop_first=True,
    dtype=int
)
data.head()
```

Output:

	Pclass	Age	SibSp	Parch	Fare	Survived	Sex_male	Embarked_Q	Embarked_S
0	3	22.0	1	0	7.2500	0	1	0	1
1	1	38.0	1	0	71.2833	1	0	0	0
2	3	26.0	0	0	7.9250	1	0	0	1
3	1	35.0	1	0	53.1000	1	0	0	1
4	3	35.0	0	0	8.0500	0	1	0	1

Step 8: Separate input features and output**Code:**

```
X= data.drop('Survived', axis=1)
y=data['Survived']
```

```
print('Input features :')
print(X.columns.tolist())
print('\nTarget: Survived')
```

Output:

```
... Input features :
['Pclass', 'Age', 'SibSp', 'Parch', 'Fare', 'Sex_male', 'Embarked_Q', 'Embarked_S']

Target: Survived
```

Step 9: Split dataset into training and testing data**Code:**

```
# X = features, y = target variable
X_train, X_test, y_train, y_test = train_test_split(
X, y, test_size=0.2, random_state=42, stratify=y)
print("Training Samples : ",len(X_train))
print("Testing Samples : ",len(X_test))
```

Output:

```
... Training Samples : 712
Testing Samples : 179
```

Step 10: Standardize the features (Important for KNN algorithm)**Code:**

```
scalar=StandardScaler()  
X_train_scaled=scalar.fit_transform(X_train)  
X_test_scaled=scalar.transform(X_test)  
  
print("Feature Standardization Completed.")
```

Output:

```
Feature Standardization Completed.
```

Step 11: Create a logistic regression model**Code:**

```
log_model=LogisticRegression(max_iter=1000)  
print(log_model)
```

Output:

```
LogisticRegression(max_iter=1000)
```

Step 12: Train logistic regression model**Code:**

```
log_model.fit(X_train_scaled,y_train)  
print("Logistic Regression Training Completed")
```

Output:

```
... Logistic Regression Training Completed
```

Step 13: Make logistic regression predictions**Code:**

```
y_pred_log= log_model.predict(X_test_scaled)
```

```
print("First 20 predictions : ")
print(y_pred_log[:20])
```

Output:

```
... First 20 predictions :
[0 0 0 0 1 0 1 0 0 0 0 0 1 0 0 0 0 0 0 1]
```

Step 14: Calculations and validations**Code:**

```
# Calculate logistic regression accuracy
accuracy_log=accuracy_score(y_test,y_pred_log)
print("Logistic Regression accuracy : ",accuracy_log)

# Logistic regression confusion matrix
cm_log =confusion_matrix(y_test, y_pred_log)
print(cm_log)

# Logistic regression classification report

print(classification_report(
    y_test,y_pred_log,
    target_names=['Did not Survive','Survived']
))
```

Output:

```
Logistic Regression accuracy : 0.8044692737430168
[[98 12]
 [23 46]]
```

	precision	recall	f1-score	support
Did not Survive	0.81	0.89	0.85	110
Survived	0.79	0.67	0.72	69
accuracy			0.80	179
macro avg	0.80	0.78	0.79	179
weighted avg	0.80	0.80	0.80	179

Explanation of Logistic Regression Evaluation Results

1. Logistic Regression Accuracy

- The Logistic Regression accuracy obtained is 0.804469, which is approximately 80.45%.
- The test dataset contains 179 passengers. From these, 144 passengers were correctly classified by the model.
- The accuracy is calculated using:

$$\text{Accuracy} = (\text{Correct Predictions} / \text{Total Predictions})$$

$$= (98 + 46) / 179$$

$$= 144 / 179$$

$$= 0.804469$$

- Therefore, the model correctly predicted the survival status of about 80.45% of the passengers in the test dataset.
- The remaining 35 passengers were classified incorrectly.

2. Logistic Regression Confusion Matrix

- The confusion matrix obtained is: $\begin{bmatrix} 98 & 12 \\ 23 & 46 \end{bmatrix}$

- The rows represent the actual class and the columns represent the predicted class.
- 98 passengers who actually did not survive were correctly predicted as did not survive.
- 12 passengers who actually did not survive were predicted as survived.
- 23 passengers who actually survived were predicted as did not survive.
- 46 passengers who actually survived were correctly predicted as survived.
- Therefore, the model made $98 + 46 = 144$ correct predictions and $12 + 23 = 35$ incorrect predictions.
- This also explains the accuracy obtained in the previous calculation.

3. Logistic Regression Classification Report

- The classification report gives precision, recall, F1-score and support for the two classes: Did not Survive and Survived.

- For the "Did not Survive" class:

- Precision = 0.81 means that out of all passengers predicted as not surviving, about 81% actually did not survive.
- Recall = 0.89 means that out of all passengers who actually did not survive, the model correctly identified about 89% of them.
- F1-score = 0.85 gives a combined measure of precision and recall for the "Did not Survive" class.
- Support = 110 means that there are 110 passengers in the test dataset who actually did not survive.

- For the "Survived" class:

- Precision = 0.79 means that out of all passengers predicted as surviving, about 79% actually survived.
 - Recall = 0.67 means that out of all passengers who actually survived, the model correctly identified about 67% of them.
 - F1-score = 0.72 gives a combined measure of precision and recall for the "Survived" class.
 - Support = 69 means that there are 69 passengers in the test dataset who actually survived.
- From these values, we can understand that the Logistic Regression model identified passengers who did not survive more effectively than passengers who survived.
- This can also be observed from the recall values. The recall for "Did not Survive" is 0.89, while the recall for "Survived" is 0.67.
- Hence, although the model achieved an overall accuracy of 80.45%, it was better at identifying passengers who did not survive than those who survived.

Part B: K-Nearest Neighbors (KNN)**Step 1: Create KNN Model****Code:**

```
from sklearn.neighbors import KNeighborsClassifier
knn_model = KNeighborsClassifier(n_neighbors=5)
print(knn_model)
```

Output:

```
KNeighborsClassifier()
```

Step 2: Train KNN Model**Code:**

```
knn_model.fit(X_train_scaled, y_train)
```

Output:

```
▼ KNeighborsClassifier ⓘ ?  
KNeighborsClassifier()
```

Step 3: Make KNN Predictions**Code:**

```
y_pred_knn = knn_model.predict(X_test_scaled)  
print("First 20 predictions:")  
print(y_pred_knn[:20])
```

Output:

```
First 20 predictions:  
[0 0 0 0 1 0 1 0 1 0 0 0 0 0 1 0 0 0 0 1]
```

Step 4: Calculate KNN Accuracy**Code:**

```
accuracy_knn = accuracy_score(y_test, y_pred_knn)  
print("KNN Accuracy:", accuracy_knn)
```

Output:

```
KNN Accuracy: 0.8156424581005587
```

Step 5: KNN Confusion Matrix

Code:

```
cm_knn = confusion_matrix(y_test, y_pred_knn)
print(cm_knn)
```

Output:

```
[[98 12]
 [21 48]]
```

Step 6: KNN Classification Report

Code:

```
print(classification_report(
    y_test, y_pred_knn,
    target_names=['Did not survive', 'Survived']
))
```

Output:

	precision	recall	f1-score	support
Did not survive	0.82	0.89	0.86	110
Survived	0.80	0.70	0.74	69
accuracy			0.82	179
macro avg	0.81	0.79	0.80	179
weighted avg	0.81	0.82	0.81	179

Explanation of KNN Evaluation Results

1. KNN Accuracy

- The KNN accuracy obtained is 0.815642, which is approximately 81.56%.
- The test dataset contains 179 passengers. From these, 146 passengers were correctly classified by the KNN model.
- The accuracy is calculated using:

Accuracy = (Correct Predictions / Total Predictions)

$$= (98 + 48) / 179$$

$$= 146 / 179$$

$$= 0.815642$$

- Therefore, the KNN model correctly predicted the survival status of about 81.56% of the passengers in the test dataset.
- The remaining 33 passengers were classified incorrectly.

2. KNN Confusion Matrix

- The confusion matrix obtained is: $\begin{bmatrix} 98 & 12 \\ 21 & 48 \end{bmatrix}$
- The rows represent the actual class and the columns represent the predicted class.
- 98 passengers who actually did not survive were correctly predicted as did not survive.
- 12 passengers who actually did not survive were predicted as survived.
- 21 passengers who actually survived were predicted as did not survive.
- 48 passengers who actually survived were correctly predicted as survived.
- Therefore, the model made $98 + 48 = 146$ correct predictions and $12 + 21 = 33$ incorrect predictions.
- This also explains the accuracy obtained in the previous calculation.

3. KNN Classification Report

- The classification report gives precision, recall, F1-score and support for the two classes: Did not Survive and Survived.
- **For the "Did not Survive" class:**
 - Precision = 0.82 means that out of all passengers predicted as not surviving, about 82% actually did not survive.
 - Recall = 0.89 means that out of all passengers who actually did not survive, the model correctly identified about 89% of them.
 - F1-score = 0.86 gives a combined measure of precision and recall for the "Did not Survive" class.
 - Support = 110 means that there are 110 passengers in the test dataset who actually did not survive.
- **For the "Survived" class:**
 - Precision = 0.80 means that out of all passengers predicted as surviving, about 80% actually survived.

- Recall = 0.70 means that out of all passengers who actually survived, the model correctly identified about 70% of them.
 - F1-score = 0.74 gives a combined measure of precision and recall for the "Survived" class.
 - Support = 69 means that there are 69 passengers in the test dataset who actually survived.
- From these values, we can understand that the KNN model identified passengers who did not survive more effectively than passengers who survived.
 - This can also be observed from the recall values. The recall for "Did not Survive" is 0.89, while the recall for "Survived" is 0.70.
 - Hence, the KNN model achieved an overall accuracy of 81.56%, and its performance was better for identifying passengers who did not survive than those who survived.
-

Step 7: Compare Logistic Regression and KNN

Code:

```
comparison = pd.DataFrame({  
    'Algorithm': ['Logistic Regression', 'KNN'],  
    'Accuracy': [accuracy_log, accuracy_knn]  
})  
comparison
```

Output:

	Algorithm	Accuracy
0	Logistic Regression	0.804469
1	KNN	0.815642

Explanation of Logistic Regression and KNN Comparison Results

1. Accuracy Comparison

- The Logistic Regression model obtained an accuracy of 0.804469, which is approximately 80.45%.
- The KNN model obtained an accuracy of 0.815642, which is approximately 81.56%.
- Both models were evaluated on the same test dataset of 179 passengers.
- Logistic Regression correctly classified 144 passengers, while KNN correctly classified 146 passengers.
- Therefore, KNN correctly classified 2 more passengers than Logistic Regression.

- The difference in accuracy is: $81.56\% - 80.45\% = 1.11\%$
- Hence, in our experiment, KNN produced a slightly higher accuracy than Logistic Regression on the Titanic test dataset.

2. Comparison of Confusion Matrices

- Logistic Regression confusion matrix: $\begin{bmatrix} 98 & 12 \\ 23 & 46 \end{bmatrix}$
- KNN confusion matrix: $\begin{bmatrix} 98 & 12 \\ 21 & 48 \end{bmatrix}$
- Both models correctly predicted 98 passengers who actually did not survive.
- Both models incorrectly predicted 12 passengers who actually did not survive as survived.
- For the passengers who actually survived, Logistic Regression correctly identified 46 passengers, while KNN correctly identified 48 passengers.
- Similarly, Logistic Regression incorrectly classified 23 actual survivors as not surviving, while KNN made 21 such incorrect predictions.
- Therefore, the main difference between the two models is in identifying the "Survived" class. KNN correctly identified 2 additional survivors and reduced the number of missed survivors from 23 to 21.
- This difference of 2 predictions is also responsible for KNN having 146 correct predictions compared with 144 for Logistic Regression.

3. Comparison of Classification Reports

- For the "Did not Survive" class:

Logistic Regression:

- Precision = 0.81
- Recall = 0.89
- F1-score = 0.85

KNN:

- Precision = 0.82
- Recall = 0.89
- F1-score = 0.86

- The recall is the same for both models at 0.89, meaning both models correctly identified a similar proportion of the passengers who actually did not survive.

- KNN has a slightly higher precision and F1-score for this class.
- For the "Survived" class:

Logistic Regression:

- Precision = 0.79
- Recall = 0.67
- F1-score = 0.72

KNN:

- Precision = 0.80
- Recall = 0.70
- F1-score = 0.74

- KNN has slightly higher precision, recall and F1-score for the "Survived" class.
- The support values are the same for both models because both models are evaluated on the same test dataset. There are 110 passengers who actually did not survive and 69 passengers who actually survived.
- From these values, we can observe that KNN performed slightly better than Logistic Regression on this particular test dataset. The improvement is mainly seen in identifying passengers who actually survived, where KNN correctly identified 48 passengers compared with 46 using Logistic Regression.
- Overall, both models performed similarly, but KNN achieved a slightly higher accuracy and slightly better precision, recall and F1-scores in this experiment.